Sistema de Telemetría y Análisis Estadístico para un Videojuego en Pygame

María Eugenia Fonseca, Isabella Rodríguez, Nicolas Montenegro, Asmaa Tarabein
Departamento de Ingeniería de Sistemas
Universidad del Norte, Barranquilla, Colombia

***Resumen*—Este trabajo presenta un sistema de telemetría para el videojuego Blue Knight, desarrollado en Pygame, que registra los eventos de cada partida en archivos de bitácora y los analiza sin cargarlos completos en memoria. Se describen el formato de la bitácora, la escritura con búfer y rotación de archivos, el tratamiento de líneas corruptas y de sesiones incompletas, y un motor estadístico que separa las métricas calculables en una sola pasada de las que requieren ordenar los datos. En los experimentos, un búfer de 100 líneas redujo el tiempo de escritura de 2.077 s a 0.116 s frente a escribir línea por línea, y la lectura con generador mantuvo cerca de 33 KB de memoria sin importar el tamaño del archivo.**

***Palabras clave*—telemetría, videojuegos, bitácoras, búfer, estadística descriptiva**

I. Introducción

En un videojuego, la telemetría es el registro automático de lo que hace el jugador y de lo que ocurre en la partida: cuándo inicia una sesión, qué nivel alcanza, cuánto puntaje obtiene o cuándo termina. Estos datos permiten entender cómo se juega, detectar niveles demasiado difíciles y encontrar errores.

El problema es que un juego con muchos jugadores genera millones de eventos, y las bitácoras resultantes pueden ser tan grandes que no caben en la memoria del computador. Cargar todo el archivo de una vez volvería el programa muy lento o lo detendría, por lo que los datos deben leerse línea por línea y escribirse por bloques.

El objetivo de este trabajo es construir un sistema en Python que escriba y lea bitácoras de eventos de forma eficiente. Para escribir se usa un búfer y varios archivos de tamaño acotado. Para leer se usa un generador que procesa un evento a la vez, ignora las líneas dañadas y cuenta las sesiones incompletas. Además, se mide cómo cambia el tiempo de escritura según el tamaño del búfer, para elegir un valor adecuado.

II. Diseño del sistema

II-A. Formato de la bitácora

Cada evento se guarda en una línea de texto con cuatro campos separados por "|": el tiempo, el identificador de la sesión, el nombre del evento y sus parámetros en la forma clave=valor separados por ";". Por ejemplo: `1760400000.5|1760399900|LEVEL_UP|level=2;time=100.5`. El juego registra seis tipos de evento: `SESSION_START`, `LEVEL_UP`, `ENEMY_KILL`, `DAMAGE_TAKEN`, `PLAYER_DEATH` y `SESSION_END`. Se eligió este formato porque es simple, se puede leer a simple vista y permite procesar el archivo línea por línea. Agregar un evento solo requiere escribir una línea al final.

II-B. Búfer y rotación de archivos

La clase `EscrituraBitacora` guarda las líneas en una lista en memoria (el búfer) y, cuando esta llega a N líneas, las escribe juntas y la vacía. El método `cerrar` escribe lo que haya quedado pendiente. Después de cada vaciado se revisa el tamaño del archivo, y si supera el umbral de 512000 bytes se abre uno nuevo con el siguiente número (`events_001.log`, `events_002.log`, etc.), construyendo las rutas con `os.path.join` [3], [4]. Se usa un búfer porque escribir en disco evento por evento es lento, y se rota el archivo para que ninguno crezca demasiado. Como el tamaño se revisa solo tras vaciar el búfer, un archivo puede pasarse un poco del umbral, lo cual se aceptó porque no afecta el funcionamiento.

II-C. Líneas corruptas

La función `parse_line` rechaza una línea si no tiene exactamente cuatro campos, si el tiempo no es un número, si la sesión o el evento están vacíos, si algún parámetro no tiene la forma clave=valor, o si campos como `score`, `level` o `duration` no son numéricos. Estas líneas se ignoran, sin detener el programa, y se cuentan en el reporte como "Lineas corruptas". Así una línea dañada no afecta el análisis de las correctas, y el conteo indica cuántos datos se perdieron.

II-D. Sesiones incompletas

Un diccionario marca cada sesión como no terminada al leer `SESSION_START` y como terminada al leer `SESSION_END`. Al final se cuentan las que quedaron sin terminar. No se eliminan sus eventos, porque son datos válidos y la sesión pudo cortarse por un cierre inesperado del juego. Solo se reportan, para que quien analice los resultados sepa que esa información está incompleta.

III. Metodología estadística

El motor estadístico divide las métricas en dos grupos: las que se calculan en una sola pasada con memoria constante, $O(1)$, y las que requieren ordenar o almacenar los valores, $O(N)$, para hallar posiciones relativas.

Para una variable con N observaciones x_i extraídas de los eventos, la media y la varianza muestral, que divide entre $N-1$ para que el estimador sea insesgado, son

$$\bar{x} = \frac{1}{N}\sum_{i=1}^{N} x_i, \tag{1}$$

$$s^2 = \frac{\sum x_i^2 - (\sum x_i)^2 / N}{N-1}, \tag{2}$$

con desviación estándar $s = \sqrt{s^2}$. El coeficiente de variación mide la dispersión relativa y permite comparar variables con distintas unidades:

$$CV = \frac{s}{\bar{x}}. \tag{3}$$

La mediana, los cuartiles Q_1 y Q_3 y el percentil 90 se calculan con los datos ordenados de forma ascendente $x_{(0)} \leq \cdots \leq x_{(N-1)}$. Para un percentil $p \in [0,1]$ se toma la posición $h = p\,(N-1)$ y el valor

$$Q(p) = \tfrac{1}{2}\left(x_{(\lfloor h \rfloor)} + x_{(\lceil h \rceil)}\right), \tag{4}$$

una variante simple dentro de las definiciones de cuantil descritas en [1]. El rango intercuartílico y los límites del criterio de Tukey [2] para valores atípicos son

$$\begin{aligned} IQR &= Q_3 - Q_1, \\ L_{inf} &= Q_1 - 1.5\,IQR, \\ L_{sup} &= Q_3 + 1.5\,IQR. \end{aligned} \tag{5}$$

Un valor fuera de $[L_{inf}, L_{sup}]$ se marca como atípico. Como segundo criterio se usa la puntuación Z, que mide la distancia a la media en desviaciones estándar y marca como atípico todo valor con $z > 3$:

$$z_i = \frac{|x_i - \bar{x}|}{s}. \tag{6}$$

Finalmente, la correlación de Pearson entre la duración x y el puntaje y de cada sesión se calcula con sumatorias:

$$r = \frac{N\sum xy - \sum x \sum y}{\sqrt{\left(N\sum x^2 - (\sum x)^2\right)\left(N\sum y^2 - (\sum y)^2\right)}}. \tag{7}$$

Las medidas (1) a (3), junto con el mínimo y el máximo, usan memoria $O(1)$ porque solo acumulan contadores y sumas. La correlación (7) guarda los valores de cada sesión, por lo que usa $O(S)$, con S el número de sesiones. La mediana, los cuartiles y el percentil 90 de (4) y (5) necesitan ordenar los datos, con memoria $O(N)$.

IV. Resultados

IV-A. *Experimento A: tamaño del búfer*

Se midió el tiempo de escritura con $N \in \{1, 10, 100, 1000\}$ y cinco repeticiones por valor (Tabla I y Fig. 1). [COMPLETAR: número de eventos escritos en cada repetición]

IV-B. *Experimento C: generador frente a lista completa*

Se comparó la lectura con generador y la carga completa en una lista, con bitácoras de distinto tamaño (Tabla II).

TABLA I
TIEMPO DE ESCRITURA SEGÚN EL TAMAÑO DEL BÚFER (5 REPETICIONES)

N	Promedio (s)	Mínimo (s)	Máximo (s)
1	2.077	2.038	2.134
10	0.314	0.281	0.349
100	0.116	0.093	0.132
1000	0.093	0.087	0.101

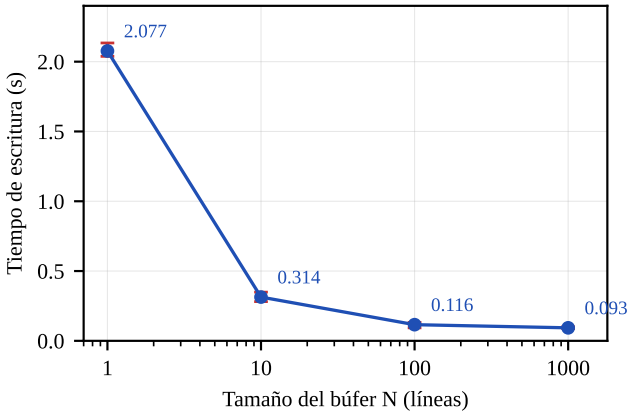


Fig. 1. Tiempo de escritura frente al tamaño del búfer en el Experimento A. Las barras indican el mínimo y el máximo de las repeticiones.

IV-C. *Experimento B y bitácora real*

[COMPLETAR: resultados del Experimento B y estadísticas (3) a (7) sobre la bitácora real de al menos 25 sesiones: mediana, cuartiles, atípicos, curva de progresión, correlación y comparación por dificultad]

V. Discusión

En el Experimento A, pasar de $N = 1$ a $N = 100$ redujo el tiempo de escritura de 2.077 s a 0.116 s, unas 18 veces menos, porque con $N = 1$ el programa accede al disco por cada evento. Entre $N = 100$ y $N = 1000$ la mejora es pequeña y los rangos de las repeticiones se solapan (Tabla I), por lo que la diferencia no es concluyente. Se descartaron $N = 1$ y $N = 10$ por su bajo rendimiento y se eligió $N = 100$, que ofrece casi todo el beneficio de velocidad. Un búfer menor también limita la pérdida de datos si el juego se cierra de forma inesperada, ya que solo se pierden las líneas aún no escritas. La memoria del búfer es pequeña en ambos casos, así que el argumento principal es ese riesgo y no el ahorro de RAM.

En el Experimento C, la memoria del generador se mantuvo cerca de 33 KB sin importar el tamaño del archivo, mientras que la lista creció de forma proporcional hasta casi 9.7 MB con 100000 eventos (Tabla II). Esto confirma que el generador es la opción adecuada cuando la bitácora puede superar la memoria disponible.

VI. Conclusiones

El sistema escribe y lee bitácoras de forma eficiente. Un búfer de 100 líneas redujo de manera considerable el tiempo

TABLA II
LECTURA CON GENERADOR FRENTE A LISTA COMPLETA

Eventos	Tamaño (KB)	T. esc. (s)	T. lect. (s)	Mem. gen. (KB)	Mem. lista (KB)
5000	249.02	0.0060	0.0263	34.09	502.60
50000	2490.23	0.0496	0.0560	33.37	4894.67
100000	4980.47	0.0848	0.1220	33.35	9686.25

de escritura, y la lectura con generador mantiene un consumo de memoria constante. Las líneas corruptas se descartan y se cuentan, y las sesiones incompletas se conservan y se reportan, de modo que el análisis posterior conoce la calidad de los datos. Como trabajo futuro se propone vaciar el búfer al cerrar el juego, para no perder eventos cuando el jugador sale a mitad de una partida.

REFERENCIAS

[1] R. J. Hyndman and Y. Fan, "Sample quantiles in statistical packages," *The American Statistician*, vol. 50, no. 4, pp. 361–365, 1996.

[2] J. W. Tukey, *Exploratory Data Analysis*. Reading, MA, USA: Addison-Wesley, 1977.

[3] Python Software Foundation, "os.path: Common pathname manipulations," *Python Documentation*. [Online]. Available: https://docs.python.org/3/library/os.path.html

[4] "Python | os.path.join() method," *GeeksforGeeks*. [Online]. Available: https://www.geeksforgeeks.org/python-os-path-join-method/